

Unique Decimal Count

Problem Description

You are given a binary string **binary** consisting only of '0' and '1'. Consider all non-empty subsequences of the string.

- A **subsequence** is formed by deleting zero or more characters without changing the order of the remaining characters.
- The binary string may contain leading zeros.
- Each subsequence is interpreted as a binary number and converted to its decimal value.

Determine the number of **distinct decimal values** that can be formed from all such non-empty subsequences. Return the count of unique decimal values.

Constraints

- $1 \leq \text{length of } binary \leq 20$
- The string *binary* consists of characters '0' and '1' only.

Sample Case 0

Input: binary = "011"

Output: 3

Explanation: Distinct subsequences are "0", "1", "01", and "11". Their decimal representations are 0, 1, 1, and 3. The distinct decimal numbers are 0, 1, and 3.

Sample Case 1

Input: binary = "11"

Output: 2

Explanation: Subsequences are "1", "1", and "11". Their decimal representations are 1, 1, and 3. The unique values are 1 and 3.

Solution Implementation (Java)

```
import java.io.*;
import java.util.*;

class Result {
    /*
     * Complete the 'getCount' function below.
     * The function returns an INTEGER.
     * The function accepts STRING binary as parameter.
     */
    public static int getCount(String binary) {
        int n = binary.length();
        Set<Integer> uniqueValue = new HashSet<>();

        // Iterate through all 2^n - 1 non-empty subsequence masks
        for (int mask = 1; mask < (1 << n); mask++) {
            int value = 0;
            for (int i = 0; i < n; i++) {
                // If the i-th bit of mask is set, include binary[i] in
                subsequence
                if ((mask & (1 << i)) != 0) {
                    value = (value << 1) + (binary.charAt(i) - '0');
                }
            }
            uniqueValue.add(value);
        }
        return uniqueValue.size();
    }
}
```